MODUL II PEMROGRAMAN GPIO

Nayla Zazki Kirani (H1H024007)
Asisten: Arga Aryanta Indrafata
Tanggal Percobaan: 07/04/2026
TK244005-Sistem Mikrokontroler Sistem
Laboratorium Baru – Jurusan Informatika UNSOED

Abstrak

Praktikum Modul 2 ini membahas pemrograman General Purpose Input /Output (GPIO) pada mikrokontroler Arduino Uno. Percobaan pertama (Percobaan 2A) mengimplementasikan pengendalian Seven Segment Display sebagai counter heksadesimal yang menampilkan digit 0 hingga F secara berurutan menggunakan fungsi digitalWrite(). Percobaan kedua (Percobaan 2B) mengintegrasikan Push Button sebagai input digital dengan mode INPUT_PULLUP untuk mengendalikan counter yang ditampilkan pada Seven Segment Display. Hasil percobaan menunjukan bahwa program dapat berjalan sesuai spesifikasi, yaitu tampilan counter berjalan tanpa error dan push button berhasil berfungsi sebagai input control. Praktikum ini membuktikan bahwa GPIO Arduino Uno dapat dimanfaatkan secara efektif untuk menghubungkan komponen input maupun output dalam satu system terintegrasi.

Kata kunci: *GPIO, Arduino Uno, Seven Segment, Push Button, Counter Digital*.

1. PENDAHULUAN

Sistem tertanam (embedded system) merupakan fondasi dari berbagai perangkat elektronik modern, mulai dari perangkat rumah tangga hingga sistem industri. Mikrokontroler, sebagai inti dari sistem tertanam, memiliki kemampuan untuk berinteraksi langsung dengan lingkungan fisik melalui pin-pin General Purpose Input/Output (GPIO). Kemampuan ini menjadikan mikrokontroler sebagai komponen kunci dalam pengembangan sistem yang memerlukan pembacaan sensor, pengendalian aktuator, maupun antarmuka pengguna [1].

Arduino Uno, yang berbasis mikrokontroler ATmega328P, menyediakan 20 pin digital I/O yang dapat dikonfigurasi secara fleksibel sebagai input atau output. Platform ini dipilih secara luas dalam dunia pendidikan dan prototipe karena kemudahan penggunaannya, dukungan komunitas yang besar, serta ketersediaan pustaka yang kaya [2].

Praktikum Modul 2 ini bertujuan untuk mengimplementasikan pemrograman GPIO pada Arduino Uno melalui dua percobaan. Percobaan pertama berfokus pada pengendalian output berupa Seven Segment Display yang menampilkan digit heksadesimal 0 hingga F sebagai counter otomatis. Percobaan kedua mengintegrasikan Push Button sebagai input digital untuk mengendalikan counter tersebut secara interaktif. Melalui kedua percobaan ini, mahasiswa diharapkan mampu memahami prinsip kerja GPIO, menerapkan fungsi-fungsi dasar Arduino, serta merangkai komponen elektronik pada breadboard.

2. STUDI PUSTAKA

Bagian ini memuat uraian teori yang mendukung pelaksanaan praktikum Modul II tentang Pemrograman GPIO.

2.1 GENERAL PURPOSE INPUT/OUTPUT (GPIO)

GPIO (General Purpose Input/Output) adalah antarmuka digital serbaguna pada mikrokontroler yang memungkinkan pengguna mengonfigurasi setiap pin sebagai input atau output sesuai kebutuhan. Berbeda dengan pin khusus yang memiliki fungsi tetap, pin GPIO dapat diprogram ulang secara dinamis melalui perangkat lunak [1]. Pada Arduino Uno berbasis ATmega328P, terdapat 20 pin digital I/O, terdiri dari 14 pin digital berlabel 0–13 dan 6 pin analog (A0–A5) yang dapat difungsikan sebagai pin digital dengan nomor 14–19 [3].

Fungsi-fungsi utama yang digunakan dalam pemrograman GPIO pada Arduino antara lain:

- pinMode(pin, mode) – Mengatur mode pin sebagai INPUT, OUTPUT, atau INPUT_PULLUP.
- digitalWrite(pin, value) – Mengirimkan sinyal digital HIGH (5V) atau LOW (0V) ke pin output.

digitalRead(pin) – Membaca nilai digital dari pin input, mengembalikan HIGH atau LOW.

2.2 Seven Segment Display

Seven Segment Display adalah komponen elektronik yang terdiri dari tujuh segmen LED (ditandai a, b, c, d, e, f, g) beserta satu segmen titik desimal (dp) yang tersusun dalam pola tertentu untuk menampilkan angka dan beberapa karakter alfabet. Setiap segmen dinyalakan atau dimatikan secara independen untuk membentuk digit yang diinginkan. Terdapat dua konfigurasi utama, yaitu Common Cathode (CC) di mana semua katoda segmen terhubung ke ground sehingga segmen menyala ketika diberi logika HIGH, dan Common Anode (CA) di mana semua anoda terhubung ke VCC sehingga segmen menyala ketika diberi logika LOW [4].

Untuk menampilkan satu digit heksadesimal (0–F), diperlukan pola kombinasi HIGH/LOW pada ketujuh segmen. Sebagai contoh, angka "0" ditampilkan dengan menyalakan segmen a, b, c, d, e, dan f, sedangkan segmen g dimatikan. Pola-pola ini biasanya disimpan dalam array lookup table pada program mikrokontroler untuk efisiensi kode [2].

2.3 Push Button dan Debouncing

Push Button adalah saklar sementara yang menutup atau membuka sirkuit selama ditekan. Dalam sistem digital, push button digunakan sebagai input diskret. Salah satu tantangan utama penggunaan push button adalah fenomena "bouncing", yaitu kondisi di mana kontak mekanik saklar memantul beberapa kali dalam waktu sangat singkat (beberapa milidetik) sebelum menstabilkan nilainya. Hal ini dapat menyebabkan pembacaan input yang tidak akurat oleh mikrokontroler [5].

Arduino menyediakan mode INPUT_PULLUP pada fungsi pinMode() yang mengaktifkan resistor pull-up internal (sekitar 20–50 kΩ). Dengan mode ini, pin akan terbaca HIGH ketika tombol tidak ditekan dan LOW ketika ditekan, tanpa memerlukan resistor eksternal tambahan. Teknik edge detection (mendeteksi perubahan state dari HIGH ke LOW atau sebaliknya) digunakan untuk memastikan satu penekanan tombol hanya mengeksekusi aksi satu kali [3].

3. METODOLOGI

Bagian ini menjelaskan komponen, alat, dan prosedur pelaksanaan setiap percobaan pada Praktikum Modul II Pemrograman GPIO.

3.1 Alat dan Bahan

Alat dan bahan yang digunakan pada kedua percobaan adalah sebagai berikut:

- 1 unit Arduino Uno (ATmega328P)
- 1 unit Seven Segment Display (Common Cathode)
- 8 buah Resistor 220 Ohm
- 1 unit Breadboard
- Kabel Jumper secukupnya
- 1 buah Push Button (untuk Percobaan 2B)
- Laptop/PC dengan Arduino IDE terinstal

3.2 Percobaan 2A: Seven Segment Counter Otomatis

Percobaan 2A bertujuan merancang rangkaian Seven Segment Display sebagai counter heksadesimal otomatis yang menampilkan digit 0 hingga F secara berurutan. Langkah-langkah yang dilakukan adalah sebagai berikut:

1. Menyiapkan alat dan bahan.
2. Merangkai Seven Segment Display ke pin digital Arduino sesuai Tabel 3-1.
3. Membuat program Arduino dengan nama file modul2_output yang berisi lookup table pola segmen untuk digit 0–F dan fungsi loop() yang memanggil displayDigit() setiap 1000 ms.
4. Mengompilasi dan mengupload program ke Arduino Uno.
5. Mengamati dan menganalisis hasil tampilan.

Tabel 3-1 Konfigurasi Wiring Seven Segment ke Arduino (Percobaan 2A & 2B)

Segmen	Label	Pin Arduino
a	a	7
b	b	6
c	c	5
d	d	11
e	e	10
f	f	8
g	g	9

dp	titik	4
Push Button	—	Pin 2

3.3 Percobaan 2B: Counter dengan Push Button

Percobaan 2B mengintegrasikan Push Button sebagai input digital untuk mengendalikan counter pada Seven Segment Display. Langkah-langkah yang dilakukan adalah:

1. Mempertahankan rangkaian Seven Segment dari Percobaan 2A.
2. Menambahkan push button pada pin 2 Arduino dengan mode INPUT_PULLUP.
3. Membuat program Arduino dengan nama file modul2_input yang mengimplementasikan edge detection untuk mendeteksi penekanan tombol dan menaikkan nilai counter.
4. Mengompilasi dan mengupload program ke Arduino Uno.

Mengamati dan menganalisis fungsi counter serta responsivitas push button.

4. HASIL DAN ANALISIS

Bagian ini menyajikan hasil dan analisis dari setiap percobaan beserta jawaban pertanyaan praktikum yang tercantum pada Modul II.

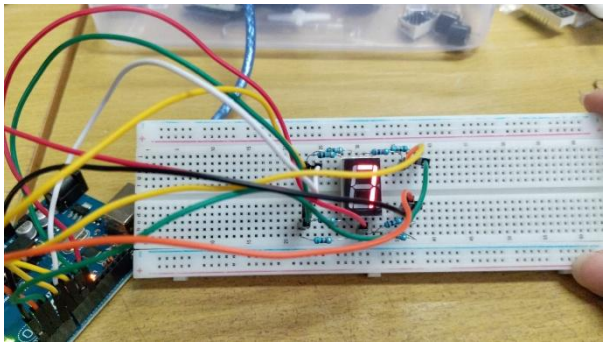
4.1 Uraian Hasil Tugas Praktikum pada Setiap Percobaan

Berikut adalah uraian hasil yang diperoleh dari masing-masing percobaan yang telah dilaksanakan pada Modul 2.

a. Percobaan 2A – Seven Segment Counter Otomatis

Pada percobaan ini berhasil dirangkai dan diprogram sebuah Seven Segment Display Common Cathode yang terhubung ke pin digital Arduino Uno (pin 4–11) melalui resistor 220Ω pada masing-masing segmen. Program modul2_output dikembangkan menggunakan pendekatan lookup table berupa array dua dimensi digitPattern[16][8] yang menyimpan pola kombinasi HIGH/LOW untuk masing-masing 16 digit heksadesimal (0–F).

Hasil yang diperoleh adalah Seven Segment Display menampilkan digit 0 hingga F secara berurutan dengan interval waktu 1 detik per digit, kemudian kembali ke digit 0 dan berulang tanpa henti. Seluruh segmen menyala dengan benar sesuai pola digit yang diinginkan tanpa ada segmen yang mati atau tampilan yang salah. Hasil ini sesuai dengan seluruh spesifikasi yang ditetapkan pada modul praktikum, yaitu konfigurasi pin sesuai petunjuk, display berjalan dari 0 sampai F secara berurutan, pola berjalan terus-menerus tanpa error, dan tidak ada LED yang mati atau tidak sesuai urutan [2].

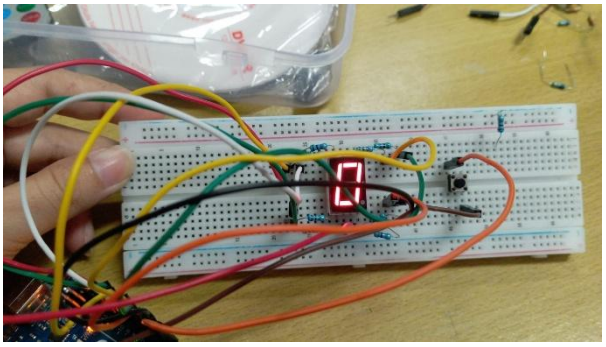


Gambar 4-1 Hasil Percobaan 2A – Seven Segment Menampilkan Counter 0–F

b. Percobaan 2B – Counter dengan Push Button

Pada percobaan ini rangkaian dari Percobaan 2A diperluas dengan menambahkan satu buah Push Button yang terhubung ke Pin 2 Arduino dengan mode INPUT_PULLUP. Program modul2_input menggabungkan pembacaan input digital dari push button dengan pengendalian output Seven Segment Display dalam satu sistem terintegrasi.

Hasil yang diperoleh menunjukkan bahwa counter berhasil dikendalikan secara interaktif menggunakan push button. Setiap satu kali penekanan tombol, nilai counter bertambah satu langkah dan Seven Segment Display menampilkan nilai yang diperbarui secara instan. Ketika counter mencapai nilai F (15), penekanan berikutnya menyebabkan counter kembali ke 0 (wrap-around). Teknik edge detection yang diimplementasikan melalui pembandingan state tombol saat ini (upState) dengan state sebelumnya (lastUpState) berhasil mencegah counter bertambah lebih dari satu kali per penekanan. Tampilan pada Seven Segment tidak mengalami glitch atau kesalahan selama pengujian, sehingga hasil ini sesuai dengan seluruh spesifikasi percobaan [3].



Gambar 4-2 Hasil Percobaan 2B – Counter Dikendalikan Push Button

4.2 Prinsip Kerja Seven Segment Display dalam Menampilkan Angka dan Karakter

Seven Segment Display adalah komponen tampilan yang tersusun dari delapan buah LED yang disusun dalam pola tertentu: tujuh segmen berbentuk batang (dilabeli a, b, c, d, e, f, g) dan satu segmen titik desimal (dp). Kedelapan segmen ini membentuk angka 8 ketika semuanya menyala, dan dengan mengaktifkan atau menonaktifkan kombinasi segmen tertentu, dapat dibentuk digit 0–9 dan karakter A–F [4].

Terdapat dua konfigurasi Seven Segment Display berdasarkan cara penyambungan pin-pin internalnya. Pada konfigurasi Common Cathode (CC), seluruh katoda LED segmen dihubungkan menjadi satu titik dan disambungkan ke GND. Segmen akan menyala apabila pin yang bersangkutan diberi logika HIGH (5V pada Arduino). Pada konfigurasi Common Anode (CA), seluruh anoda LED segmen dihubungkan ke VCC, dan segmen akan menyala apabila pin-nya diberi logika LOW (0V). Pada praktikum ini digunakan konfigurasi Common Cathode, sehingga logika 1 (HIGH) pada array digitPattern berarti segmen menyala dan logika 0 (LOW) berarti padam [4].

Prinsip kerja menampilkan digit melibatkan tiga tahapan utama sebagai berikut:

1. Penentuan pola segmen. Untuk setiap digit yang akan ditampilkan, ditentukan segmen mana saja yang harus menyala. Sebagai contoh, digit 0 memerlukan segmen a, b, c, d, e, f menyala (nilai 1) dan segmen g padam (nilai 0), sehingga polanya adalah {1,1,1,1,1,1,0,0}. Digit 1 hanya memerlukan segmen b dan c menyala, dengan pola {0,1,1,0,0,0,0,0}. Pola untuk seluruh 16 digit heksadesimal disimpan dalam lookup table pada program [2].
2. Pengiriman sinyal melalui GPIO. Mikrokontroler Arduino membaca pola dari lookup table, kemudian mengirimkan sinyal HIGH atau LOW ke masing-masing pin GPIO yang tersambung ke segmen a–g dan dp menggunakan fungsi digitalWrite(). Setiap pin GPIO bertugas mengendalikan satu segmen secara independen.
3. Pembatasan arus. Resistor 220Ω dipasang seri antara pin Arduino dan masing-masing segmen LED untuk membatasi arus yang mengalir. Tanpa resistor, arus yang terlalu besar dapat merusak LED maupun pin mikrokontroler. Nilai 220Ω menghasilkan arus sekitar 13,6 mA per segmen, yang berada dalam batas aman operasi LED [1].

Dengan memanfaatkan prinsip di atas, Seven Segment Display mampu menampilkan tidak hanya angka desimal 0–9, tetapi juga karakter heksadesimal A–F dengan mengaktifkan kombinasi segmen yang sesuai. Karakter seperti A (segmen a, b, c, e, f, g), b (segmen c, d, e, f, g), dan seterusnya dibentuk dari pola-pola yang sudah didefinisikan secara eksplisit dalam program [2].

4.3 Cara Kerja Sistem Counter pada Program

Sistem counter pada kedua program (modul2_output dan modul2_input) bekerja berdasarkan mekanisme yang berbeda, namun keduanya menggunakan prinsip dasar yang sama yaitu pengelolaan variabel pencacah yang nilainya dipetakan ke tampilan Seven Segment Display melalui lookup table.

a. Counter Otomatis (modul2_output)

Pada program Percobaan 2A, sistem counter bekerja secara penuh otomatis berbasis waktu. Fungsi loop() Arduino yang berjalan terus-menerus mengeksekusi sebuah for loop dengan variabel iterator i yang dimulai dari 0 dan berakhir di 15 (kondisi i < 16). Setiap iterasi memanggil displayDigit(i) untuk menampilkan pola digit ke-i, kemudian memanggil delay(1000) untuk memberikan jeda selama 1.000 milidetik. Setelah iterasi ke-15 selesai, for loop berakhir, fungsi loop() kembali dipanggil dari awal oleh Arduino secara otomatis, sehingga counter mulai lagi dari 0 dan membentuk siklus tak terhingga [2].

Proses di dalam fungsi displayDigit(int num) adalah membaca baris ke-num dari array digitPattern, kemudian menggunakan for loop internal untuk mengiterasi 8 elemen (indeks 0–7 mewakili segmen a, b, c, d, e, f, g, dp), dan mengirimkan nilai elemen tersebut (HIGH atau LOW) ke pin GPIO yang sesuai melalui digitalWrite(segmentPins[i], digitPattern[num][i]). Dengan cara ini, seluruh delapan segmen diperbarui setiap kali satu digit baru ditampilkan.

b. Counter Berbasis Input Push Button (modul2_input)

Pada program Percobaan 2B, sistem counter bekerja secara event-driven berbasis kejadian penekanan tombol. Variabel global currentDigit menyimpan nilai counter saat ini dan diinisialisasi dengan nilai 0. Di dalam fungsi loop() yang berjalan terus-menerus, program membaca kondisi push button melalui digitalRead(btnUp) pada setiap siklusnya [3].

Kunci dari sistem ini adalah teknik edge detection. Program tidak hanya memeriksa apakah tombol sedang ditekan (upState == LOW), melainkan memeriksa apakah terjadi transisi dari tidak ditekan ke ditekan, yaitu kondisi lastUpState == HIGH && upState == LOW. Kondisi ini hanya bernilai benar sesaat ketika tombol baru saja ditekan, tepat pada siklus pertama di mana state tombol berubah dari HIGH ke LOW. Dengan demikian, meskipun pengguna menahan tombol selama beberapa detik yang menghasilkan ratusan iterasi loop(), counter hanya bertambah satu kali saja.

Alur lengkap sistem counter pada Percobaan 2B adalah sebagai berikut:

1. Pada setiap iterasi loop(), program membaca state tombol saat ini ke variabel upState melalui digitalRead(btnUp).
2. Program membandingkan upState dengan lastUpState, yaitu state tombol pada iterasi sebelumnya.
3. Jika terdeteksi transisi HIGH ke LOW (penekanan tombol baru), currentDigit dinaikkan 1.
4. Jika currentDigit melebihi 15, nilainya direset ke 0 (wrap-around), sehingga counter bersifat sirkular pada rentang 0–15.
5. Fungsi displayDigit(currentDigit) dipanggil untuk memperbarui tampilan Seven Segment Display sesuai nilai counter terbaru.
6. Nilai lastUpState diperbarui dengan nilai upState saat ini untuk digunakan pada perbandingan iterasi berikutnya.

Mekanisme wrap-around memastikan counter tidak pernah keluar dari rentang valid (0–15), yang sekaligus mencegah akses memori di luar batas array digitPattern. Secara keseluruhan, sistem counter ini mendemonstrasikan pola desain polling input yang efisien dan umum digunakan pada sistem tertanam untuk menangani input diskret [5].

5. KESIMPULAN

Dari praktikum Modul 2 Pemrograman GPIO, dapat disimpulkan bahwa:

1. GPIO pada Arduino Uno dapat dikonfigurasi secara fleksibel sebagai input maupun output menggunakan fungsi pinMode(), digitalWrite(), dan digitalRead(), memungkinkan interaksi langsung dengan komponen elektronik seperti Seven Segment Display dan Push Button.
2. Percobaan 2A berhasil mengimplementasikan Seven Segment Display Common Cathode sebagai counter heksadesimal 0–F menggunakan lookup table pola segmen. Counter berjalan secara otomatis dengan interval 1 detik per digit tanpa error dan sesuai spesifikasi.
3. Percobaan 2B berhasil mengintegrasikan Push Button dengan mode INPUT_PULLUP untuk mengendalikan counter secara interaktif. Teknik edge detection yang diimplementasikan memastikan satu penekanan tombol hanya menghasilkan satu perubahan nilai counter, mencegah pembacaan ganda akibat bouncing.
4. Pemahaman terhadap konfigurasi Common Cathode vs Common Anode sangat penting karena menentukan logika penulisan pola segmen. Kesalahan konfigurasi akan menyebabkan tampilan yang terbalik atau tidak berfungsi.

Praktikum ini memperkuat pemahaman tentang prinsip dasar sistem tertanam: interaksi antara perangkat lunak (program Arduino) dan perangkat keras (komponen elektronik) melalui antarmuka GPIO..

DAFTAR PUSTAKA

[1] M. Barr dan A. Massa, *Programming Embedded Systems: With C and GNU Development Tools*, 2nd ed. Sebastopol: O'Reilly Media, 2006.

[2] M. Margolis, B. Jepson, dan N. R. Weldin, *Arduino Cookbook*, 3rd ed. Sebastopol: O'Reilly Media, 2020.

[3] R. Pease, "Seven-segment display decoder/driver," *IEEE Spectrum*, vol. 47, no. 4, pp. 42–43, Apr. 2010, doi: 10.1109/MSPEC.2010.5434109.

[4] S. Monk, *Programming Arduino: Getting Started with Sketches*, 2nd ed. New York: McGraw-Hill Education, 2016.

[5] J. Ganssle dan M. Barr, *Embedded Systems Dictionary*. Lawrence: CMP Books, 2003